Example 3-11

Write a program to count up from 00 to FFH and send the count to SFR of Port B. Use one CALL subroutine for sending the data to Port B and another one for time delay. Put a time delay in between each issuing of data to Port B.

Solution:

Example 3-11

Write a program to count up from 00 to FFH and send the count to SFR of Port B. Use one CALL subroutine for sending the data to Port B and another one for time delay. Put a time delay in between each issuing of data to Port B.

Solution:

```
        list P=PIC18F458
#include P18F458.INC

COUNT   EQU     0x07    ;use location 07 for count-up
MYREG   EQU     0x08    ;use location 08 for delay

ORG     0
        MOVLW   0                   ;WREG = 0
        MOVWF   COUNT               ;count = 0
BACK    CALL    DISPLAY
        GOTO    BACK

;------- increment and put it in PORTB
DISPLAY INCF    COUNT,F         ;increment count
        MOVFF   COUNT,PORTB     ;send it to PORTB
        CALL    DELAY
        RETURN                  ;return to caller

;------- this is the delay subroutine
ORG     300H    ;put time delay at address 300H
DELAY   MOVLW   0xFF            ;WREG = 255, the counter
        MOVWF   MYREG
AGAIN   NOP     ;no operation wastes clock cycles
        NOP
        NOP
        DECF    MYREG,F
        BNZ     AGAIN  ;repeat until MYREG becomes 0
        RETURN                  ;return to caller
        END                     ;end of asm file
```

Example 3-12

Rewrite the main part of Example 3-9 as efficiently as you can.

Solution:

Example 3-12

Rewrite the main part of Example 3-9 as efficiently as you can.

Solution:

```
        MYREG EQU 0x08
        ORG   0
        MOVLW 0x55          ;load WREG with 55H
        MOVWF PORTB                 ;issue value in PORTB SFR
BACK    RCALL DELAY         ;time delay
        COMPF PORTB,F               ;complement Port B SFR
        BRA   BACK          ;keep doing this indefinitely
;————————this is the delay subroutine
DELAY   MOVLW 0xFF          ;WREG = 255, the counter
        MOVWF MYREG
AGAIN   NOP                 ;no operation wastes clock cycles
        NOP
        DECF  MYREG,F
        BNZ   AGAIN         ;repeat until MYREG becomes 0
        RETURN              ;return to caller (MYREG = 0)
        END                 ;end of asm file
```

SECTION 3.3: PIC18 TIME DELAY AND INSTRUCTION PIPELINE

Instruction cycle time for the PIC

It takes a certain amount of time for the CPU to execute an instruction. In the PIC, this time is referred to as *instruction cycles* (referred to as machine cycles in some other CPUs).

In the PIC family, the length of the instruction cycle depends on the frequency of the oscillator connected to the PIC system. The crystal oscillator, along with on-chip circuitry, provide the clock source for the PIC CPU (see Chapter 8). In the PIC18, one instruction cycle consists of four oscillator periods. Therefore, to calculate the instruction cycle for the PIC, we take 1/4 of the crystal frequency, then take its inverse, as shown in Example 3-14.

If the crystal frequency = 8 Hz, it means the crystal produces 8 oscillations (clock cycles) every second.

Step 1: Find instruction frequency

For PIC18:

$$F_{instruction} = \frac{F_{crystal}}{4}$$

$$= \frac{8}{4} = 2 \text{ Hz}$$

So the PIC has **2 instruction cycles per second.**

Step 2: Find instruction-cycle time

$$T_{instruction} = \frac{1}{F_{instruction}}$$

$$= \frac{1}{2} = 0.5 \text{ seconds}$$

Answer:

$$\boxed{T_{instruction} = 0.5 \text{ seconds}}$$

Or simply:

8 Hz crystal → 4 oscillator periods per instruction → 2 instruction cycles/second → each instruction cycle = 0.5 s.

Example 3-14

The following shows the crystal frequency for three different PIC-based systems. Find the period of the instruction cycle in each case.
(a) 4 MHz (b) 16 MHz (c) 20 MHz

Solution:
(a) 4/4 = 1 MHz; instruction cycle is 1/1 MHz = 1 μs (microsecond)
(b) 16 MHz/4 = ?
(c) 20 MHz/4 = ?

Example 3-14

The following shows the crystal frequency for three different PIC-based systems. Find the period of the instruction cycle in each case.
(a) 4 MHz (b) 16 MHz (c) 20 MHz

Solution:
(a) 4/4 = 1 MHz; instruction cycle is 1/1 MHz = 1 μs (microsecond)
(b) 16 MHz/4 = 4 MHz; instruction cycle = 1/4 MHz = 0.25 μs = 250 ns (nanosecond)
(c) 20 MHz/4 = 5 MHz; instruction cycle = 1/5 MHz = 0.2 μs = 200 ns

Example 3-15

For a PIC18 system of 4 MHz, find how long it takes to execute each of the following instructions:

```
(a) MOVLW      (b) DECF       (c) MOVWF
(d) ADDLW      (e) NOP        (f) GOTO
(g) CALL       (h) BNZ
```

Solution:

The machine cycle for a system of 4 MHz is 1 µs, as shown in Example 3-14. Appendix A shows instruction cycles for each of the above instructions. Therefore, we have:

Instruction	*Instruction cycles*	*Time to execute*
(a) MOVLW 0x55	1	1 × 1 µs = 1 µs
(b) DECF MYREG	1	1 × 1 µs = 1 µs
(c) MOVWF	1	1 × 1 µs = 1 µs
(d) ADDLW	1	1 × 1 µs = 1 µs
(e) NOP	1	1 × 1 µs = 1 µs
(f) GOTO	2	2 × 1 µs = 2 µs
(g) CALL	2	2 × 1 µs = 2 µs
(h) BNZ	2/1	(2 µs taken, 1 µs if it falls through)

Example 3-16

Find the size of the delay of the code snippet below if the crystal frequency is 4 MHz:

Solution:

From Appendix A, we have the following machine cycles for each instruction of the DELAY subroutine:

```
                                    Instruction Cycle
MYREG EQU   0x08          ;use location 08 as counter

DELAY    MOVLW    0xFF                 1
         MOVWF    MYREG                1

AGAIN    NOP                           1
         NOP                           1
         DECF     MYREG,F              1
         BNZ      AGAIN                2

         RETURN                        1
```

Example 3-16

Find the size of the delay of the code snippet below if the crystal frequency is 4 MHz:

Solution:

From Appendix A, we have the following machine cycles for each instruction of the DELAY subroutine:

```
                                    Instruction Cycle
MYREG EQU   0x08          ;use location 08 as counter

DELAY    MOVLW    0xFF              1
         MOVWF    MYREG             1

AGAIN    NOP                        1
         NOP                        1
         DECF     MYREG,F           1
         BNZ      AGAIN             2

         RETURN                     1
```

Therefore, we have a time delay of $[(255 \times 5) + 1 + 1 + 1] \times 1\ \mu s = 1278\ \mu s$.
Notice that BNZ takes two instruction cycles if it jumps back, and takes only one when falling through the loop. That means the above number should be 1277 μs.

Example 3-18

Find the size of the delay in the following program if the crystal frequency is 4 MHz:

```
MYREG EQU   0x08                ;use location 08 as counter

          ORG       0
BACK      MOVLW     0x55        ;load WREG with 55H
          MOVWF     PORTB       ;send 55H to port B
          CALL      DELAY       ;time delay
          MOVLW     0xAA        ;load WREG with AA (in hex)
          MOVWF     PORTB       ;send AAH to port B
          CALL      DELAY
          GOTO      BACK        ;keep doing this indefinitely

;-------  this is the delay subroutine
          ORG       300H        ;put time delay at address 300H
DELAY     MOVLW     0xFA        ;WREG = 250, the counter
          MOVWF     MYREG
AGAIN     NOP                   ;no operation wastes clock cycles
          NOP
          NOP
          DECF      MYREG, F
          BNZ       AGAIN       ;repeat until MYREG becomes 0
          RETURN                ;return to caller
          END                   ;end of asm file
```

Solution:

Example 3-18

Find the size of the delay in the following program if the crystal frequency is 4 MHz:

```
MYREG EQU   0x08              ;use location 08 as counter

          ORG       0
BACK      MOVLW     0x55      ;load WREG with 55H
          MOVWF     PORTB     ;send 55H to port B
          CALL      DELAY     ;time delay
          MOVLW     0xAA      ;load WREG with AA (in hex)
          MOVWF     PORTB     ;send AAH to port B
          CALL      DELAY
          GOTO      BACK      ;keep doing this indefinitely

;-------  this is the delay subroutine
          ORG       300H      ;put time delay at address 300H
DELAY     MOVLW     0xFA      ;WREG = 250, the counter
          MOVWF     MYREG
AGAIN     NOP                 ;no operation wastes clock cycles
          NOP
          NOP
          DECF      MYREG, F
          BNZ       AGAIN     ;repeat until MYREG becomes 0
          RETURN              ;return to caller
          END                 ;end of asm file
```

Solution:

From Appendix A, we have the following machine cycles for each instruction of the DELAY subroutine:

```
                                  Instruction Cycle

DELAY     MOVLW     0xFA              1
          MOVWF     MYREG             1

AGAIN     NOP                         1
          NOP                         1
          NOP                         1
          DECF      MYREG, F          1
          BNZ       AGAIN             2

          RETURN                      1
```

Therefore, we have a time delay of $[(250 \times 6) + 1 + 1 + 1] \times 1\ \mu s = 1503\ \mu s$.

Loop inside a loop delay

Another way to get a large delay is to use a loop inside a loop, which is also called a *nested loop*. See Example 3-18. Compare that with Example 3-19 to see the disadvantage of using many NOPs.

Example 3-18

For a instruction cycle of 1 μs, find the time delay in the following subroutine:

```
R2      EQU     0x7
R3      EQU     0x8
DELAY                                   Instruction Cycle
                MOVLW       D'200'      1
                MOVWF       R2          1
AGAIN           MOVLW       D'250'      1
                MOVWF       R3          1
HERE            NOP                     1
                NOP                     1
                DECF        R3, F       1
                BNZ         HERE        2
                DECF        R2, F       1
                BNZ         AGAIN       2
                RETURN                  1
```

Subroutine Overview

This PIC assembly subroutine `DELAY` is designed to generate a **fixed time delay** by using **nested loops**. The program depends on the instruction cycle time (here $1\mu s$).

Registers:

- **R2 = 0x07** → Outer loop counter.
- **R3 = 0x08** → Inner loop counter.

Step 1: Inner Loop (`HERE`)

- Inside `HERE`, the instructions executed are:

```scss
NOP (1 µs)
NOP (1 µs)
DECF R3,F (1 µs)
BNZ HERE (2 µs when branching, 1 µs when falling through)
```

Copy code

So each iteration of the `HERE` loop costs **5 cycles = 5 µs**.

- Since R3 = 250, we repeat this loop **250 times**.

$$\text{Time for inner loop} = 5 \times 250 \times 1\mu s = 1250\mu s$$

Step 2: Outer Loop (`AGAIN`)

- Outer loop repeats the inner loop **200 times**, because R2 = 200.
- Without overhead:

$$200 \times 1250\mu s = 250,000\mu s$$

Step 3: Overhead of Outer Loop Instructions

Each time we enter/repeat the `AGAIN` loop, extra instructions add overhead:

- `MOVLW D'250'` → 1 µs
- `MOVWF R3` → 1 µs
- `DECF R2,F` → 1 µs
- `BNZ AGAIN` → 2 µs

So, **5 µs per outer iteration × 200 iterations = 1000 µs.**

Step 4: Adjustment for Final Branch

When the `BNZ HERE` falls through (inner loop ends), **200 µs** must be subtracted (since BNZ costs 1 µs instead of 2 on final fall-through).

Step 5: Total Delay

$$250,000 + 1000 - 200 = 250,800\mu s = 250.8ms$$

Example 3-19

Find the time delay for the following subroutine, assuming a crystal frequency of 4 MHz. Discuss the disadvantage of this over Example 3-18.

Find time delay for AGAIN loop only.

```
MYREG EQU   0x8
                                        Machine Cycle

DELAY   MOVLW   D'200'                      1
        MOVWF   MYREG                       1

AGAIN   NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        NOP                                 1
        DECF    MYREG, F                    1
        BNZ     AGAIN                       2

        RETURN                              1
```

Solution:

The time delay inside the AGAIN loop is [200(13 + 2)] × 1 μs = 3000 μs. NOP is a 2-byte instruction, even though it does not do anything except to waste cycle time. There are 17 instructions in the above DELAY program, and all the instructions are 2-byte instructions. This means the loop delay takes 34 bytes of ROM code space, and gives us only a 3000 μs delay. That is the reason we use a nested loop instead of NOP instructions to create a time delay. Chapter 9 shows how to use PIC timers to create delays much more efficiently.

Example 3-20

Write a program to toggle all the bits of SFR PORTB every 1 s. Assume that the crystal frequency is 10 MHz and the system is using a PIC18F458.

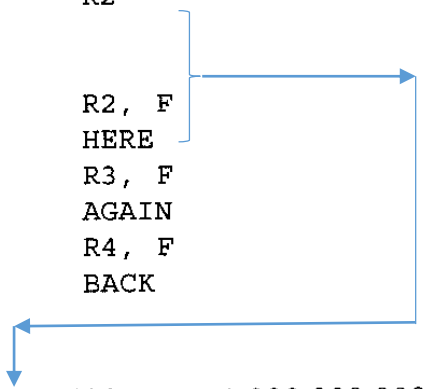
Solution:

Solution:

```
;tested using MPLAB with PIC18F458 operating at 10 MHz
R2      EQU     0x2
R3      EQU     0x3
R4      EQU     0x4


        MOVLW   0x55          ;load WREG with 55H
        MOVWF   PORTB         ;send 55H to PORTB B
BACK    CALL    DELAY_500MS   ;time delay
        COMF    PORTB         ;complement PORTB
        GOTO    BACK          ;keep doing this indefinitely

;-------  this is the delay subroutine
DELAY_500MSEC
            MOVLW       D'20'
            MOVWF       R4
BACK        MOVLW       D'100'
            MOVWF       R3
AGAIN       MOVLW       D'250'
            MOVWF       R2
HERE        NOP
            NOP
            DECF        R2, F
            BNZ         HERE
            DECF        R3, F
            BNZ         AGAIN
            DECF        R4, F
            BNZ         BACK
            RETURN
```



Delay $20 \times 100 \times 250 \times 5 \times 400$ ns $= 1{,}000{,}000{,}000$ ns $= 1{,}000{,}000$ µs $= 1$ s.

In this calculation, we have not included the overhead associated with the two outer loops. Use the MPLAB simulator to verify the delay.

CHAPTER 4

PIC I/O PORT PROGRAMMING

SECTION 4.2: I/O BIT MANIPULATION PROGRAMMING

In this section we further examine the PIC18 I/O instructions. We pay special attention to I/O bit manipulation because it is a powerful and widely used feature of the PIC family.

I/O ports and bit-addressability

Sometimes we need to access only 1 or 2 bits of the port instead of the entire 8 bits. A powerful feature of PIC I/O ports is their capability to access individual bits of the port without altering the rest of the bits in that port. For all PIC ports, we can access either all 8 bits or any single bit without altering the rest. Table 4-8 lists the single-bit instructions for the PIC18. Although the instructions in Table 4-9 can be used for any registers in the data RAM file register, I/O port operations use them most often. We will see the use of these instructions throughout future chapters.

Table 4-8: Single-Bit (Bit-Oriented) Instructions for PIC18

Instruction	Function
BSF fileReg,bit	Bit Set fileReg (set the bit: bit = 1)
BCF fileReg,bit	Bit Clear fileReg (clear the bit: bit = 0)
BTG fileReg,bit	Bit Toggle fileReg (complement the bit)
BTFSC fileReg,bit	Bit test fileReg, skip if clear (skip next instruction if bit = 0)
BTFSS fileReg,bit	Bit test fileReg, skip if set (skip next instruction if bit = 1)

Table 4-9: Single-Bit Addressability of Ports for PIC18F458/4580

PORT	PORTB	PORTC	PORTD	PORTE	Port Bit
RA0	RB0	RC0	RD0	RE0	D0
RA1	RB1	RC1	RD1	RE1	D1
RA2	RB2	RC2	RD2	RE2	D2
RA3	RB3	RC3	RD3		D3
RA4	RB4	RC4	RD4		D4
RA5	RB5	RC5	RD5		D5
	RB6	RC6	RD6		D6
	RB7	RC7	RD7		D7

BSF (bit set fileReg)

To set HIGH a single bit of a given fileReg, we use the syntax "`BSF fileReg, bit_num`" where fileReg can be any location in the file register and bit_num is the desired bit number from 0 to 7. Although the bit-oriented instructions can be used for manipulation of bits D0–D7 of any file register, they are mostly used for I/O ports in embedded systems. For example, "`BSF PORTB, 5`" sets HIGH bit 5 of Port B.

Example 4-2

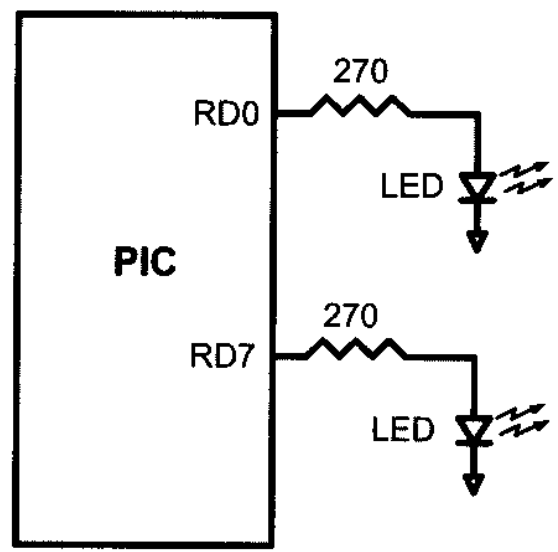
An LED is connected to each pin of Port D. Write a program to turn on each LED from pin D0 to pin D7. Call a delay module before turning on the next LED.

Example 4-2

An LED is connected to each pin of Port D. Write a program to turn on each LED from pin D0 to pin D7. Call a delay module before turning on the next LED.

Solution:

```
CLRF   TRISD        ;make PORTD an output port
BSF    PORTD,0      ;bit set turns on RD0
CALL   DELAY        ;delay before next one
BSF    PORTD,1      ;turn on RD1
CALL   DELAY        ;delay before next one
BSF    PORTD,2
CALL   DELAY
BSF    PORTD,3
CALL   DELAY
BSF    PORTD,4
CALL   DELAY
BSF    PORTD,5
CALL   DELAY
BSF    PORTD,6
CALL   DELAY
BSF    PORTD,7
CALL   DELAY
```



BCF (bit clear fileReg)

To clear a single bit of a given fileReg, we use the syntax "`BCF fileReg, bit_number`". Remember that for I/O ports, we must activate the appropriate bit in the TRISx register if we want the pin to reflect the changes. For example, the following code toggles pin RB2 continuously:

```
      BCF   TRISB, 2          ;bit = 0, make RB2 an output pin
AGAIN BSF   PORTB, 2          ;bit set (RB2 = high)
      CALL  DELAY
      BCF   PORTB, 2          ;bit clear(RB2 = low)
      CALL  DELAY
      BRA   AGAIN
```

Example 4-3

Write the following programs:

(a) Create a square wave of 50% duty cycle on bit 0 of Port C.

(b) Create a square wave of 66% duty cycle on bit 3 of Port C.

Example 4-3

Write the following programs:
(a) Create a square wave of 50% duty cycle on bit 0 of Port C.
(b) Create a square wave of 66% duty cycle on bit 3 of Port C.

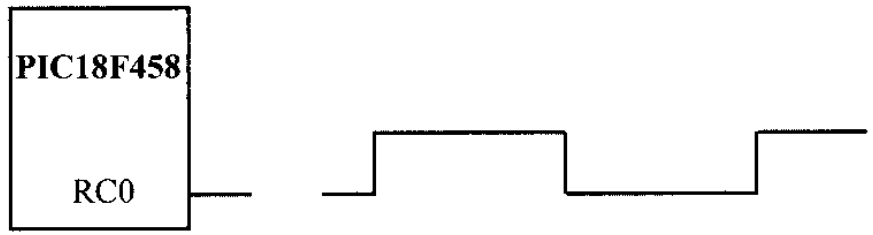
Solution:

(a) The 50% duty cycle means that the "on" and "off" states (or the high and low portions of the pulse) have the same length. Therefore, we toggle RC0 with a time delay between each state.

```
      BCF   TRISC,0     ;clear TRIS bit for RC0 = out
HERE  BSF   PORTC,0     ;set to HIGH RC0 (RC0 = 1)
      CALL  DELAY       ;call the delay subroutine
      BCF   PORTC,0     ;RC0 = 0
      CALL  DELAY
      BRA   HERE        ;keep doing it
```

Another way to write the above program is:

```
      BCF   TRISC,0     ;make RC0 = out
HERE  BTG   PORTC,0     ;complement bit 0 of PORTC
      CALL  DELAY       ;call the delay subroutine
      BRA   HERE        ;keep doing it
```



(b) A 66% duty cycle means that the "on" state is twice the "off" state.

```
        BCF    TRISC,3      ;clear TRISC3 bit for output
BACK    BSF    PORTC,3      ;RC3 = 1
        CALL   DELAY        ;call the delay subroutine
        CALL   DELAY        ;twice for 66%
        BCF    PORTC,3      ;RC3 = 0
        CALL   DELAY        ;call delay once for 33%
        BRA    BACK         ;keep doing it
```



BTG (bit toggle fileReg)

To toggle a single bit of a given fileReg, we use the syntax "`BTG fileReg, bit_number`".

```
      BCF   TRISB, 2          ;make RB2 an output pin
BACK  BTG   PORTB, 2          ;toggle pin RB2 only
      CALL  DELAY
      BRA   BACK
```

Notice that RB2 is the third bit of Port B (the first bit is RB0, the second bit is RB1, etc.). This is shown in Table 4-9. See Example 4-2 for an example of bit manipulation of I/O bits.

Notice in Example 4-2 that unused portions of Port C are undisturbed. This single-bit addressability of I/O ports is one of most powerful features of the PIC microcontroller and is one of the reasons that many designers choose the PIC over other microcontrollers. We will see the use of the bit-addressability of I/O ports in future chapters.

Checking an input pin

To make decisions based on the status of a given bit in the file register, we use the instructions BTFSC (bit test fileReg skip if clear) and BTFSS (bit test fileReg skip if set). These single-bit instructions are widely used for I/O operations. They allow you to monitor a single pin and make a decision depending on whether it is 0 or 1. Again it must be noted that the instructions BTFSC and BTFSS can be used for any bits of the file register, including the I/O ports A, B, C, D, and so on.

Example 4-4

Write a program to perform the following:
(a) Keep monitoring the RB2 bit until it becomes HIGH;
(b) When RB2 becomes HIGH, write value 45H to Port C, and also send a HIGH-to-LOW pulse to RD3.

Solution:

Example 4-4

Write a program to perform the following:
(a) Keep monitoring the RB2 bit until it becomes HIGH;
(b) When RB2 becomes HIGH, write value 45H to Port C, and also send a HIGH-to-LOW pulse to RD3.

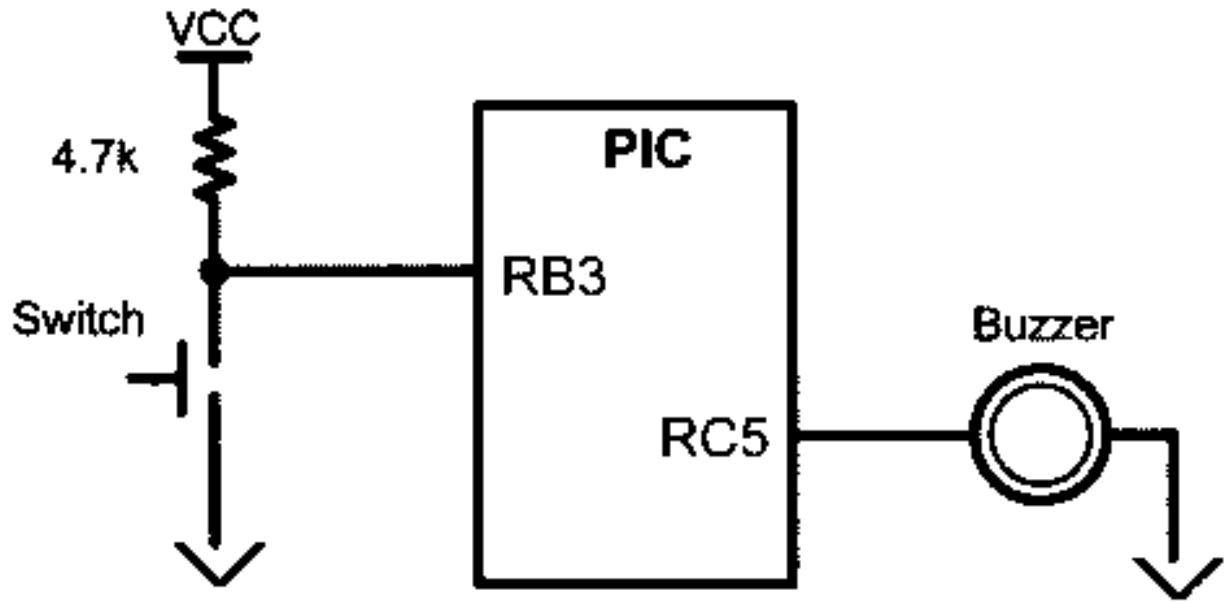
Solution:

```
      BSF   TRISB,2      ;make RB2 an input
      CLRF  TRISC        ;make PORTC an output port
      BCF   PORTD,3      ;make RD3 an output
      MOVLW 0x45         ;WREG = 45h
AGAIN BTFSS PORTB,2      ;bit test RB2 for HIGH
      BRA   AGAIN        ;keep checking if LOW
      MOVWF PORTC        ;issue WREG to Port C
      BSF   PORTD,3      ;bit set fileReg RD3 (H-to-L)
      BCF   PORTD,3      ;bit clear fileReg RD3 (L)
```

In this program, instruction "`BTFSS  PORTB,2`" stays in the loop as long as RB2 is LOW. When RB2 becomes HIGH, it skips the branch instruction to get out of the loop, and writes the value 45H to Port C. It also sends a HIGH-to-LOW pulse to RD3.

Example 4-5

Assume that bit RB3 is an input and represents the condition of a door alarm. If it goes LOW, it means that the door is open. Monitor the bit continuously. Whenever it goes LOW, send a HIGH-to-LOW pulse to port RC5 to turn on a buzzer.

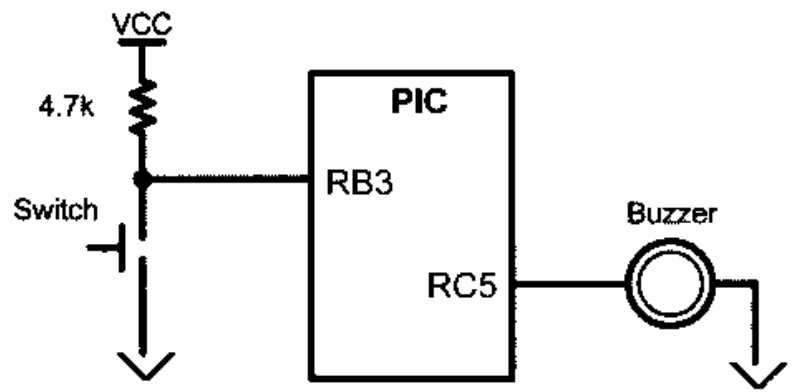


Example 4-5

Assume that bit RB3 is an input and represents the condition of a door alarm. If it goes LOW, it means that the door is open. Monitor the bit continuously. Whenever it goes LOW, send a HIGH-to-LOW pulse to port RC5 to turn on a buzzer.

Solution:

```
       BSF    TRISB,3      ;make RB3 an input
       BCF    TRISC,5      ;make RC5 an output
HERE   BTFSC  PORTB,3      ;keep monitoring RB3 for HIGH
       BRA    HERE         ;stay in the loop
       BSF    PORTC,5      ;make RC5 HIGH
       BCF    PORTC,5      ;make RC5 LOW for H-to-L
       BRA    HERE
```



Example 4-6

A switch is connected to pin RB2. Write a program to check the status of SW and perform the following:
(a) If SW = 0, send the letter 'N' to PORTD.
(b) If SW = 1, send the letter 'Y' to PORTD.

Example 4-6

A switch is connected to pin RB2. Write a program to check the status of SW and perform the following:
(a) If SW = 0, send the letter 'N' to PORTD.
(b) If SW = 1, send the letter 'Y' to PORTD.

Solution:

```
        BSF   TRISB,2      ;make RB2 an input
        CLRF  TRISD        ;make PORTD an output port
AGAIN   BTFSS PORTB,2      ;bit test RB2 for HIGH
        BRA   OVER         ;it must be LOW
        MOVLW A'Y'         ;WREG = 'Y' ASCII letter Y
        MOVWF PORTD        ;issue WREG to PORTD
        GOTO  AGAIN        ;we can use BRA too
OVER    MOVLW A'N'         ;WREG = 'N' ASCII  letter N
        MOVWF PORTD        ;issue WREG to PORTD
        GOTO  AGAIN        ;we can use BRA too
```

Example 4-7

A switch is connected to pin RB2. Write a program to check the status of SW and perform the following:
(a) If SW = 0, send letter 'N' to PORTD.
(b) If SW = 1, send letter 'Y' to PORTD.

Use the BTFSC instruction to check the SW status. This is another version of Example 4-6 using the BTFSC instruction instead of BTFSS.

Example 4-7

A switch is connected to pin RB2. Write a program to check the status of SW and perform the following:
(a) If SW = 0, send letter 'N' to PORTD.
(b) If SW = 1, send letter 'Y' to PORTD.

Use the BTFSC instruction to check the SW status. This is another version of Example 4-6 using the BTFSC instruction instead of BTFSS.

Solution:

```
       BSF   TRISB,2      ;make RB2 an input
       CLRF  TRISD        ;make PORTD an output port
AGAIN  BTFSC PORTB,2      ;bit test RB2 for LOW
       BRA   OVER         ;it must be HIGH
       MOVLW A'N'         ;WREG = 'N' ASCII letter N
       MOVWF PORTD        ;issue WREG to PORTD
       BRA   AGAIN        ;we can use GOTO
OVER   MOVLW A'Y'         ;WREG = 'Y' ASCII letter Y
       MOVWF PORTD        ;issue WREG to PORTD
       BRA   AGAIN        ;we can use GOTO
```